

Linux CLI and Debugging for Engineers

Treat the shell as a way to ask precise questions about a system. Learn navigation, quoting, streams, processes, logs, and debugging through a disposable local project. Prefer narrow read-only inspection before changing state, and preserve the evidence that supports each diagnosis.

For: Developers using Linux or WSL who want repeatable command-line workflows and a disciplined way to diagnose processes, files, permissions, and service failures.

Suggested pace: 4 weeks of short local exercises | Beginner to intermediate

What you will be able to do

- Use paths, quoting, pipelines, exit codes, and environment variables correctly.
- Inspect a process's state, logs, open files, and listening endpoints.
- Form and test a local debugging hypothesis with a minimal reproducer.
- Avoid destructive troubleshooting and accidental exposure of secrets.

Build the right mental model

Commands have contracts

Understand arguments, standard input/output/error, exit status, and working directory. Quote paths containing spaces. A pipeline can hide an earlier failure unless the shell and script handle it deliberately; visible output alone is not proof of success.

Permissions are not a reason to escalate blindly

Inspect ownership, mode, path traversal permissions, and the actual user running a process. Broad permission changes or running everything as root can hide the cause and introduce risk. Fix the narrow ownership or access requirement instead.

Debug by discrimination

State what you expect, what happened, and one observation that distinguishes competing causes. A minimal reproducer reduces irrelevant state. Keep the original error and versions so another person can verify the result instead of relying on a vague narrative.

Study with evidence.

Practice in a dedicated directory or disposable environment. Use read-only commands first, target only processes you own, and review any state-changing command before execution. Never paste credentials into a chat or a command history to get help.

Week 1: Shell fundamentals

Learn navigation, quoting, redirection, pipes, environment scope, and exit codes. Create a small script that accepts a path, validates it, and reports useful errors. Test spaces and missing files.

Deliverable and check: A script that returns meaningful status and does not confuse filenames with shell syntax.

Week 2: Files and process state

Inspect file metadata, permissions, disk usage, process trees, CPU, memory, and local listening ports. Compare the environment of an interactive shell with a launched service. Learn graceful process termination before forced termination.

Deliverable and check: A checklist separating missing file, wrong working directory, permission failure, and port conflict.

Week 3: Logs and traces

Add structured timestamps and request IDs to a local service. Use its logs, a debugger, and authorized system-call tracing to inspect a failed request. Redact secrets and avoid collecting unrelated traffic.

Deliverable and check: One reproducible failure with a short evidence timeline and a confirmed controlling operation.

Week 4: Automation and recovery

Automate the reproducer and a regression test. Handle command failures explicitly and clean only resources created by the test. Document prerequisites and platform differences rather than assuming every Linux distribution has identical tools.

Deliverable and check: A repeatable diagnosis and fix that another engineer can run without manual guessing.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Investigate a service that cannot open its configuration

A program runs from your terminal but fails when launched as a service. The error says a configuration file is missing. Avoid changing permissions or copying files until you know which path is actually being opened.

A worked approach

1. Record the process user, working directory, arguments, and configuration path. A relative path resolves against the process working directory, which may differ from the repository directory used interactively.
2. Use logs or a trace of the process you control to confirm the attempted file path and error. Inspect ownership and directory traversal permissions only after validating that the path is the intended one.
3. Pass an explicit configuration path or define a stable application-relative location. Add a test that launches from a different working directory and confirms the program either succeeds or reports a clear path error.

Expected result

The fix addresses the real environmental assumption rather than masking it with broad access changes. The regression test captures the difference between interactive and service execution.

Verify the result

- Does the observed attempted path support the hypothesis?
- Is the fix limited to the application's configuration contract?
- Does the error omit secret configuration values?

Extend the exercise

Repeat the exercise with a port already occupied by another local test process. Identify the owning process and choose an unused port instead of terminating unrelated services.

Make the learning stick.

Why did a command succeed but the script fail?

The script may use a different shell, environment, working directory, user, or quoting behavior. Compare those inputs and the exit status before assuming the tool itself is unreliable.

Should I use sudo to fix a permission error?

Only when elevated access is genuinely required and authorized. First inspect the path and ownership. Running the whole application with more privilege can create new files owned by root and make the original problem worse.

What makes a bug report useful?

Include expected and actual behavior, exact versions, a minimal reproducer, relevant sanitized output, and the smallest known conditions. Explain what you tested and what remains uncertain rather than providing only a screenshot of the final symptom.

Avoid these traps

- Deleting caches or killing processes before preserving the failure evidence.
- Using broad recursive permission changes as a default fix.
- Leaking tokens through command history, screenshots, or debug logs.

Ready for the next step?

- Shell quoting and exit codes are understood.
- A service failure can be reduced to a reproducible command.
- Inspection precedes risky state changes.
- A regression test verifies the actual root cause.

Reliable next reads

- [MIT Missing Semester](#)
- [Linux manual pages](#)
- [GNU Bash manual](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.